

AIAC Assignment 4.1

Name : P. Ashwitha

Ht.no : 2303A51163

Batch :24

Q1. Zero-Shot Prompting (Basic Lab Task)

Task:

Write a Python function that classifies a given text as Spam or Not Spam using zero-shot prompting.

Steps:

1. Construct a prompt without any examples.
2. Clearly specify the output labels.
3. Display only the predicted label.

Input:

"Congratulations! You have won a free lottery ticket."

Expected Output:

Spam

Prompt : Write a Python function that classifies a given text as Spam or Not Spam

Code :

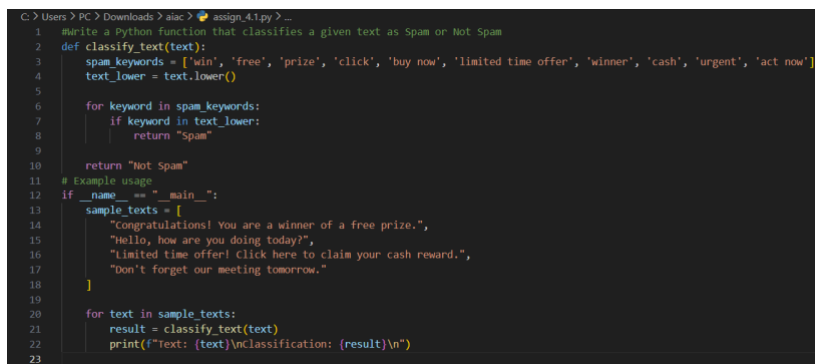
```
def classify_text(text):
    spam_keywords = ['win', 'free', 'prize', 'click', 'buy now', 'limited time offer', 'winner', 'cash', 'urgent', 'act now']
    text_lower = text.lower()

    for keyword in spam_keywords:
        if keyword in text_lower:
            return "Spam"

    return "Not Spam"

# Example usage
if __name__ == "__main__":
    sample_texts = [
        "Congratulations! You are a winner of a free prize.",
        "Hello, how are you doing today?",
        "Limited time offer! Click here to claim your cash reward.",
        "Don't forget our meeting tomorrow."
    ]

    for text in sample_texts:
        result = classify_text(text)
        print(f"Text: {text}\nClassification: {result}\n")
```



```
C:\Users> PC> Downloads> aiac> assign_4.1.py >
1 #write a Python function that classifies a given text as Spam or Not Spam
2 def classify_text(text):
3     spam_keywords = ['win', 'free', 'prize', 'click', 'buy now', 'limited time offer', 'winner', 'cash', 'urgent', 'act now']
4     text_lower = text.lower()
5
6     for keyword in spam_keywords:
7         if keyword in text_lower:
8             return "Spam"
9
10    return "Not Spam"
11
12 # Example usage
13 if __name__ == "__main__":
14     sample_texts = [
15         "Congratulations! You are a winner of a free prize.",
16         "Hello, how are you doing today?",
17         "Limited time offer! Click here to claim your cash reward.",
18         "Don't forget our meeting tomorrow."
19     ]
20
21     for text in sample_texts:
22         result = classify_text(text)
23         print(f"Text: {text}\nClassification: {result}\n")
```

Output :

Text: Congratulations! You are a winner of a free prize.
Classification: Spam

Text: Hello, how are you doing today?
Classification: Not Spam

Text: Limited time offer! Click here to claim your cash reward.
Classification: Spam

Text: Don't forget our meeting tomorrow.
Classification: Not Spam

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Text: Congratulations! You are a winner of a free prize.
Classification: Spam

Text: Hello, how are you doing today?
Classification: Not Spam

Text: Limited time offer! Click here to claim your cash reward.
Classification: Spam

Text: Don't forget our meeting tomorrow.
Classification: Not Spam
```

Q2. One-Shot Prompting (Emotion detection)

Task:

Write a Python program that detects the emotion of a sentence using one-shot prompting.

Emotions: ['happy', 'sad', 'angry', 'excited', 'nervous', 'neutral']

Steps:

1. Provide one labeled example inside the prompt.
2. Take a sentence as input.
3. Print the predicted emotion

Prompt :

```
"""
```

```
i am happy today. output:happy
```

```
"""
```

```
#write a python program that detects the emotion of a sentence and
emotions=[happy,sad,angry,excited,nervous,neutral]
```

Code :

```
def detect_emotion(sentence):
    emotions = ['happy', 'sad', 'angry', 'excited', 'nervous', 'neutral']
    for emotion in emotions:
        if emotion in sentence.lower():
            return emotion
    return "Emotion not detected"

input_sentence = input("Enter a sentence: ")
detected_emotion = detect_emotion(input_sentence)
print(f"The detected emotion is: {detected_emotion}")
```

Output:

```
Enter a sentence: i am very sad
```

```
The detected emotion is: sad
```

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiaac/assign_4.1.py
Enter a sentence: i am very sad
The detected emotion is: sad
```

Q3. Few-Shot Prompting (Student Grading Based on Marks)

Task:

Write a Python program that predicts a student's grade based on marks using few-shot prompting.

Grades:

['A', 'B', 'C', 'D', 'F']

Grading Criteria (to be inferred from examples):

- 90–100 → A
- 80–89 → B
- 70–79 → C
- 60–69 → D
- Below 60 → F

Prompt : """

90–100 → A
80–89 → B
70–79 → C
60–69 → D
Below 60 → F
"

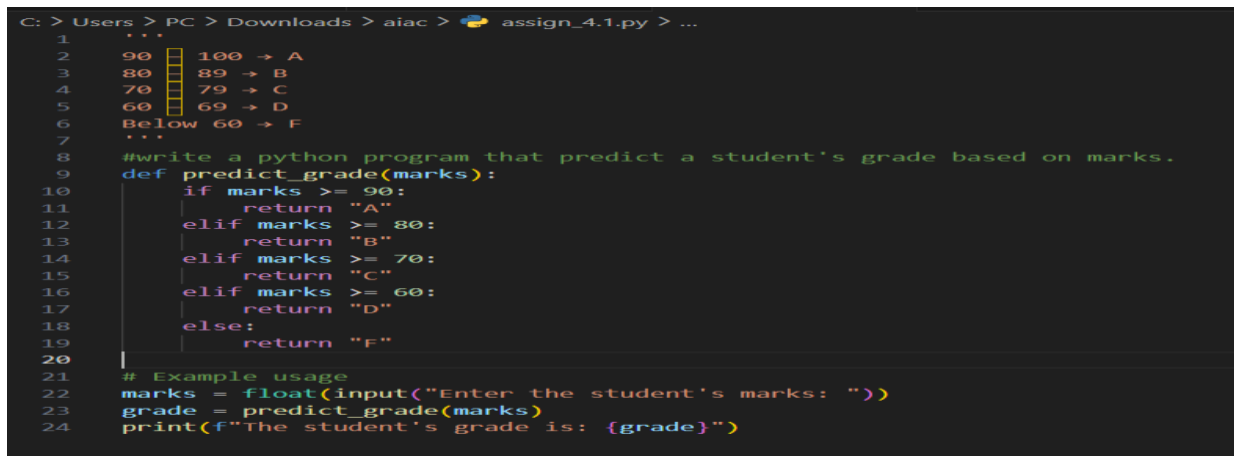
#write a python program that predict a student's grade based on marks.

Code:

```
def predict_grade(marks):  
    if marks >= 90:  
        return "A"  
    elif marks >= 80:  
        return "B"  
    elif marks >= 70:  
        return "C"  
    elif marks >= 60:  
        return "D"  
    else:  
        return "F"
```

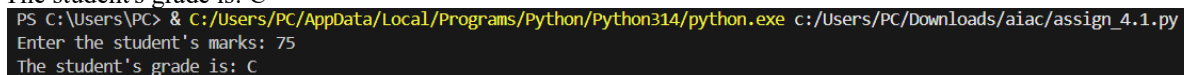
Example usage

```
marks = float(input("Enter the student's marks: "))  
grade = predict_grade(marks)  
print(f"The student's grade is: {grade}")
```

A screenshot of a code editor window with a dark background. The title bar shows the file path: "C: > Users > PC > Downloads > aiac > assign_4.1.py > ...". The code is written in Python and includes comments. Lines 1-7 show a grade scale: 90-100 to A, 80-89 to B, 70-79 to C, 60-69 to D, and below 60 to F. Line 8 is a comment: "#write a python program that predict a student's grade based on marks." Line 9 is the function definition: "def predict_grade(marks):". Lines 10-19 are the conditional logic for the grade prediction. Line 20 is a comment: "# Example usage". Lines 21-24 are the example usage code: "marks = float(input('Enter the student's marks: '))", "grade = predict_grade(marks)", and "print(f'The student's grade is: {grade}')." The code is color-coded with syntax highlighting.

Output :

Enter the student's marks: 75
The student's grade is: C

A screenshot of a terminal window with a dark background. The prompt is "PS C:\Users\PC>". The command entered is "& C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/assign_4.1.py". The output shows "Enter the student's marks: 75" and "The student's grade is: C".

Q4. Multi-Shot Prompting (Indian Zodiac Sign Prediction using Month Name)

Task:

Write a Python program that predicts a person's Indian Zodiac sign (Rashi) based on the month of birth (month name) using multi-shot prompting.

Indian Zodiac Order (Simplified Month-Based Model): The Indian Zodiac cycle starts in March with Mesha and follows this order:

March → Mesha
April → Vrishabha
May → Mithuna
June → Karka
July → Simha
August → Kanya
September → Tula
October → Vrischika
November → Dhanu

December → Makara
January → Kumbha
February → Meena

Prompt :

"""

March → Mesha
April → Vrishabha
May → Mithuna
June → Karka
July → Simha
August → Kanya
September → Tula
October → Vrischika
November → Dhanu
December → Makara
January → Kumbha
February → Meena
"""

write a python program that predicts a persons zodiac sign based on month of birth

Code :

```
def zodiac_sign(month):
    month = month.lower()
    if month == "march":
        return "Mesha"
    elif month == "april":
        return "Vrishabha"
    elif month == "may":
        return "Mithuna"
    elif month == "june":
        return "Karka"
    elif month == "july":
        return "Simha"
    elif month == "august":
        return "Kanya"
    elif month == "september":
        return "Tula"
    elif month == "october":
        return "Vrischika"
    elif month == "november":
        return "Dhanu"
    elif month == "december":
        return "Makara"
    elif month == "january":
        return "Kumbha"
    elif month == "february":
        return "Meena"
    else:
        return "Invalid month"
month = input("Enter your month of birth: ")
sign = zodiac_sign(month)
print(f"Your zodiac sign is: {sign}")
```

```
def zodiac_sign(month):
    month = month.lower()
    if month == "march":
        return "Mesha"
    elif month == "april":
        return "Vrishabha"
    elif month == "may":
        return "Mithuna"
    elif month == "june":
        return "Karka"
    elif month == "july":
        return "Simha"
    elif month == "august":
        return "Kanya"
    elif month == "september":
        return "Tula"
    elif month == "october":
        return "Vrischika"
    elif month == "november":
        return "Dhanu"
    elif month == "december":
        return "Makara"
    elif month == "january":
        return "Kumbha"
    elif month == "february":
        return "Meena"
    else:
        return "Invalid month"
month = input("Enter your month of birth: ")
sign = zodiac_sign(month)
print(f"Your zodiac sign is: {sign}")
```

Output :

Enter your month of birth: july

Your zodiac sign is: Simha

```
Enter your month of birth: july
Your zodiac sign is: Simha
```

Q5. Result Analysis Based on Marks

Task: Write a Python program that determines whether a student Passes or Fails based on marks using Chain-of-Thought (CoT) prompting.

Result Categories:

['Pass', 'Fail']

Prompt :

```
"""
```

Read the marks from the user.

based on marks determine whether student passed or not.

handle invalid inputs

```
"""
```

Code :

```
try:
```

```
    marks = float(input("Enter the marks obtained (0-100): "))
```

```
    if marks < 0 or marks > 100:
```

```
        print("Invalid input: Marks should be between 0 and 100.")
```

```
    elif marks >= 40:
```

```
        print("Student has passed.")
```

```
    else:
```

```
        print("Student has failed.")
```

```
except ValueError:
```

```
    print("Invalid input: Please enter a numeric value for marks.")
```

```
C: > Users > PC > Downloads > aia > assign_4.1.py > ...
1  """
2  Read the marks from the user.
3  based on marks determine whether student passed or not.
4  handle invalid inputs
5  """
6  try:
7      marks = float(input("Enter the marks obtained (0-100): "))
8      if marks < 0 or marks > 100:
9          print("Invalid input: Marks should be between 0 and 100.")
10     elif marks >= 40:
11         print("Student has passed.")
12     else:
13         print("Student has failed.")
14 except ValueError:
15     print("Invalid input: Please enter a numeric value for marks.")
```

Output :

Enter the marks obtained (0-100): 86

Student has passed.

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/assign_4.1.py
Enter the marks obtained (0-100): 86
Student has passed.
```

Q6 Voting Eligibility Check (Chain-of-Thought Prompting)

Task: Write a Python program that determines whether a person is eligible to vote using Chain-of-Thought (CoT) prompting.

Prompt :

```
"""
read the age of the user
print he can vote or not
handle invalid cases
"""
```

Code :

```
try:
    age = int(input("Enter your age: "))
    if age < 0:
        print("Invalid age. Age cannot be negative.")
    elif age >= 18:
        print("You are eligible to vote.")
    else:
        print("You are not eligible to vote.")
except ValueError:
    print("Invalid input. Please enter a valid age.")
```

```
C: > Users > PC > Downloads > aiac > assign_4.1.py > ...
1      """
2      read the age of the user
3      print he can vote or not
4      handle invalid cases
5      """
6      try:
7          age = int(input("Enter your age: "))
8          if age < 0:
9              print("Invalid age. Age cannot be negative.")
10         elif age >= 18:
11             print("You are eligible to vote.")
12         else:
13             print("You are not eligible to vote.")
14     except ValueError:
15         print("Invalid input. Please enter a valid age.")
```

Output:

Enter your age: 19

You are eligible to vote.

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/assign_4.1.py
Enter your age: 19
You are eligible to vote.
```

Q7 Prompt Chaining (String Processing – Palindrome Names)

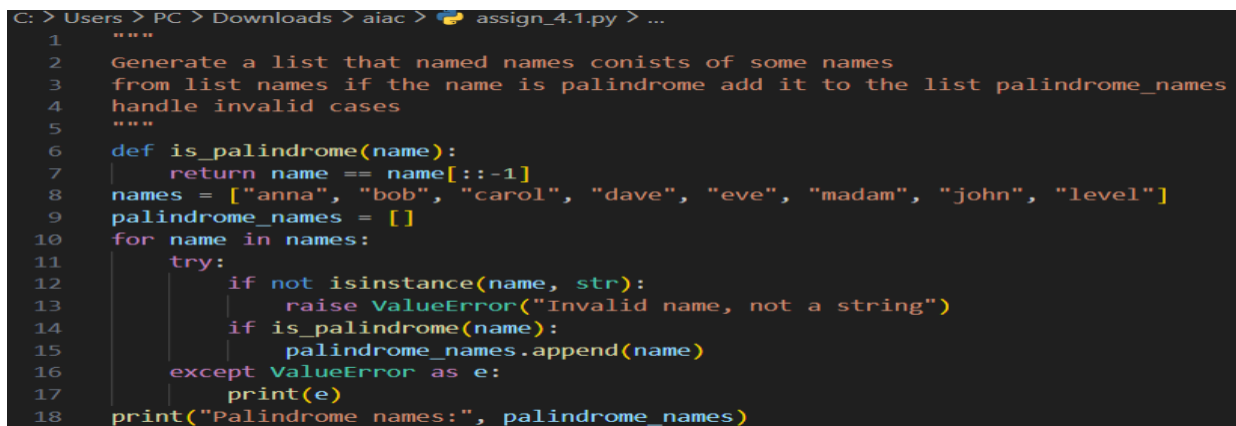
Task: Write a Python program that uses the prompt chaining technique to identify palindrome names from a list of student names.

Prompt :

```
"""
Generate a list that named names consists of some names
from list names if the name is palindrome add it to the list palindrome_names
handle invalid cases
"""
```

Code :

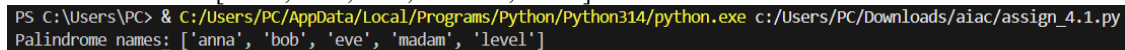
```
def is_palindrome(name):
    return name == name[::-1]
names = ["anna", "bob", "carol", "dave", "eve", "madam", "john", "level"]
palindrome_names = []
for name in names:
    try:
        if not isinstance(name, str):
            raise ValueError("Invalid name, not a string")
        if is_palindrome(name):
            palindrome_names.append(name)
    except ValueError as e:
        print(e)
print("Palindrome names:", palindrome_names)
```



```
C:\Users > PC > Downloads > aiac > assign_4.1.py > ...
1 """
2 Generate a list that named names consists of some names
3 from list names if the name is palindrome add it to the list palindrome_names
4 handle invalid cases
5 """
6 def is_palindrome(name):
7     return name == name[::-1]
8 names = ["anna", "bob", "carol", "dave", "eve", "madam", "john", "level"]
9 palindrome_names = []
10 for name in names:
11     try:
12         if not isinstance(name, str):
13             raise ValueError("Invalid name, not a string")
14         if is_palindrome(name):
15             palindrome_names.append(name)
16     except ValueError as e:
17         print(e)
18 print("Palindrome names:", palindrome_names)
```

Output :

Palindrome names: ['anna', 'bob', 'eve', 'madam', 'level']



```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/assign_4.1.py
Palindrome names: ['anna', 'bob', 'eve', 'madam', 'level']
```

Q8 Prompt Chaining (String Processing – Word Length Analysis)

Task: Write a Python program that uses prompt chaining to analyze a list of words. In the first prompt, generate a list of words. In the second prompt, traverse the list and calculate the length of each word. In the third prompt, use the output of the previous step to determine whether each word is Short (length less than 5) or Long (length greater than or equal to 5), and display the result for each word

Prompt :

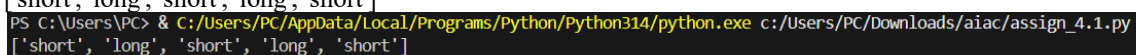
```
"""
generate a list of words named words_list
traverse the list and calculate the length of each word.
if length is less than 5 make a list and add as "short" to list
if length greater than 7 add as "long" to list
"""
```

Code :

```
words_list = ["apple", "banana", "kiwi", "strawberry", "fig", "watermelon", "pear", "grape"]
length_category = []
for word in words_list:
    if len(word) < 5:
        length_category.append("short")
    elif len(word) > 7:
        length_category.append("long")
print(length_category)
```

Output :

['short', 'long', 'short', 'long', 'short']



```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/assign_4.1.py
['short', 'long', 'short', 'long', 'short']
```